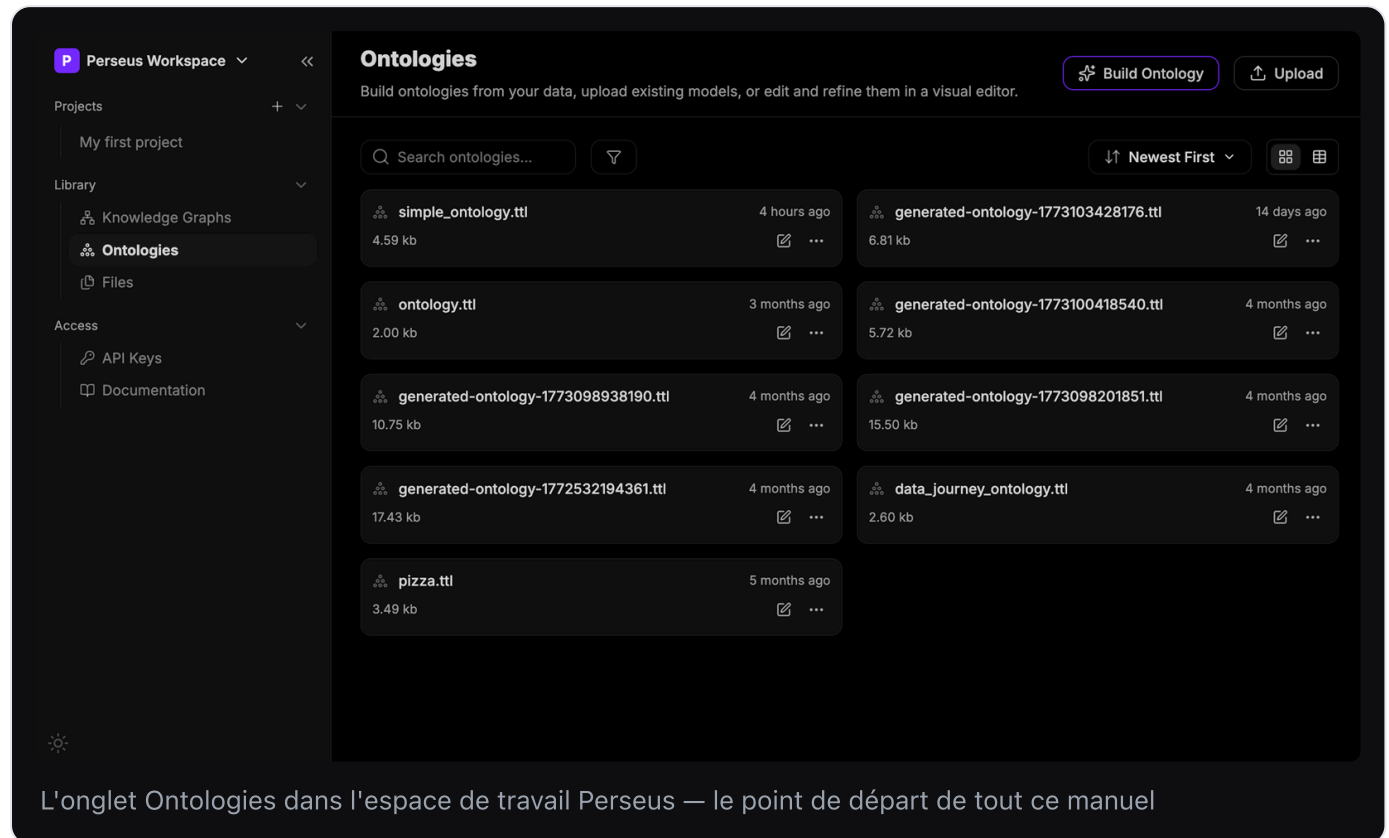


Lettria Perseus — Manuel de l'éditeur d'ontologies

Un guide de l'éditeur visuel d'ontologies de Lettria Perseus.



1. À quoi sert cet outil

Perseus extrait des données structurées à partir de documents non structurés. Laissé à lui-même, il extraira tout ce qu'il trouve. Une **ontologie**, c'est la façon dont vous lui dites ce qu'il a le *droit* de trouver : quels types de choses existent dans votre domaine, quels attributs elles portent, et comment elles peuvent être reliées entre elles.

L'éditeur d'ontologies est l'endroit où vous inspectez et réglez ce schéma à la main. Utilisez-le pour :

- Relire une ontologie que Perseus a générée depuis vos documents, et corriger ce qu'il a raté.
- Ajouter une classe ou une relation que le générateur a manquée.
- Renommer les choses pour que le graphe extrait utilise le vocabulaire de votre équipe.
- Déclarer quelques individus — valeurs d'énumération fixes, ou entités de référence — pour que l'extraction ait des points d'ancrage contre lesquels réconcilier ses instances.

Un éditeur d'ontologies est un éditeur de schéma. C'est plus proche, dans l'esprit, de l'édition d'un schéma de base de données que de l'édition de données — sauf que le « schéma » est ici lui-même exprimé sous forme de graphe.

2. Concepts et vocabulaire

Nouveau dans les ontologies ? Deux références méritent de rester ouvertes à côté de ce manuel. L'*Ontology Development 101* de Noy & McGuinness — issu de Stanford, l'une des introductions de référence du domaine — est la porte d'entrée en douceur vers les concepts ci-dessous. Pour les définitions précises et normatives, la référence OWL du W3C est le standard technique vers lequel se tourner.

Graphe

Un **graphe**, ce sont des nœuds reliés par des arêtes. Dans un graphe de connaissances, les nœuds sont des *choses* (San Francisco, Jane Doe, les USA) et les arêtes sont des *relations étiquetées* entre elles (Jane Doe `→LOCATED_IN→` San Francisco).

L'unité d'information est le **triplet** : sujet, prédicat, objet.

Jane Doe	LOCATED_IN	San Francisco
↑	↑	↑
subject	predicate	object

Tout ce qui se trouve dans l'éditeur — chaque classe, chaque propriété, chaque annotation — finit par se compiler en un tas de triplets. L'interface n'est qu'un visage plus avenant posé dessus.

Ontologie

Une **ontologie** est le schéma du graphe : le vocabulaire et les règles. Elle déclare que `Person` et `City` sont des types de choses, que `LOCATED_IN` est une arête légale, et qu'elle peut aller d'une `Person` vers une `City` (mais pas, disons, d'une `Year` vers une `Year`).

En termes relationnels, définir une classe avec ses propriétés revient grosso modo à une instruction `CREATE TABLE` . Une relation entre deux classes joue le rôle d'une clé étrangère ; un attribut portant une valeur directe joue le rôle d'une simple colonne. Une ontologie est plus expressive que cela, et elle est stockée comme donnée plutôt que comme DDL.

Une ontologie n'est *pas* vos données. Elle décrit la forme de vos données.

Classe

Une **classe** est un type — une catégorie de chose. `Person`, `City`, `Country`, `Year`. Les noms de classes sont par convention au singulier et en PascalCase.

Les classes forment une hiérarchie. `City` peut être enfant de `Place`, qui peut être enfant de `Thing`. Une classe enfant est une **sous-classe** : toute `City` est automatiquement aussi un `Place`. C'est la relation « est-un » de l'héritage objet, et comme dans certains langages objet, une classe peut avoir plusieurs parents.

Perseus affiche cette hiérarchie dans le panneau de gauche de l'onglet **Classes**, avec un chevron de dépliage sur toute classe qui a des enfants.

Individu

Un **individu** est un membre précis et nommé d'une classe, déclaré à *l'intérieur de l'ontologie elle-même*. C'est la distinction qui fait trébucher : un individu n'est pas la même chose qu'une *instance*. Une instance est une chose concrète qui vit dans le graphe de connaissances — les données extraites de vos documents. Un individu, lui, vit dans l'ontologie, et vous l'y placez délibérément, pour l'une de deux raisons.

Comme valeur d'énumération. Quand une propriété ne doit jamais prendre qu'une valeur parmi un ensemble fixe, vous déclarez ces valeurs comme des individus. La Music Ontology, par exemple, déclare `Album` comme un individu nommé de la classe `ReleaseType` :

```
mo:Album a owl:NamedIndividual, mo:ReleaseType .
```

Un graphe de connaissances *utilise* alors cet individu comme valeur d'une propriété :

```
ex:imagine a mo:MusicalWork ;  
            mo:release_type mo:Album .
```

Ici l'individu `Album`, défini une seule fois dans l'ontologie, sert de type de sortie énuméré d'une œuvre musicale.

Comme fixture de test pour un raisonneur. Vous pouvez aussi ajouter des individus pour « tester unitairement » l'ontologie : lancez un raisonneur sur une poignée d'individus écrits à la main et vérifiez qu'il infère ce que vous attendez (et signale les contradictions que vous attendez). Ces individus existent pour exercer le schéma, pas pour porter des données de production.

Dans les deux cas, l'individu fait partie du schéma que vous rédigez. La plupart des choses concrètes de votre graphe final seront des *instances* découvertes par l'extraction, pas des individus que vous avez déclarés à la main.

Propriété

Une **propriété** est une arête nommée — le prédicat d'un triplet. Vous pouvez définir autant de sortes de propriétés que nécessaire ; OWL en fournit trois par défaut, et ce sont ces trois-là que Perseus expose :

Sorte	Part de	Va vers	Exemple
Propriété d'objet (object property)	une classe	une classe	<code>LOCATED_IN</code> , <code>employs</code> , <code>manages</code>
Propriété de type de données (datatype property)	une classe	un type de données (chaîne, nombre, date)	<code>name</code> , <code>description</code> , <code>url</code>
Propriété d'annotation (annotation property)	une classe	une classe ou un type de données	<code>rdfs:label</code> , <code>rdfs:comment</code>

Ces définitions vivent dans l'*ontologie*, elles relient donc des classes et des types de données, pas des individus concrets. `Person` `—LOCATED_IN→` `City` est une définition de propriété ; le fait `Jane Doe` `—LOCATED_IN→` `San Francisco` est une seule arête du graphe de connaissances qui la respecte.

C'est le côté droit qui décide de la sorte qu'il vous faut. Si la chose à droite est une classe que vous voudriez décrire plus avant — elle a ses propres attributs, ses propres relations — c'est une **propriété d'objet**. Si ce n'est qu'une valeur, une chaîne ou un nombre, c'est une **propriété de type de données**.

`Person` `LOCATED_IN` `City` est une propriété d'objet : une `City` est une vraie entité avec ses propres faits. `Person` `name` s'étendant sur une chaîne est une propriété de type de données : la chaîne n'est qu'une chaîne. `employer` pointant vers une `Company` est une propriété d'objet ; `employerName` contenant une chaîne est une propriété de type de données. La propriété d'objet est généralement celle que vous voulez dans un graphe de connaissances — les liens sont tout l'intérêt du graphe.

Les propriétés d'annotation sont les intruses : elles portent de la documentation, pas des faits du domaine. Elles sont là pour les humains et les outils, et les raisonneurs les ignorent.

Domaine et co-domaine

Toute propriété d'objet et de type de données a un **domaine** (domain) et un **co-domaine** (range).

- **Domain** = la ou les classes que le *sujet* a le droit d'être. Le côté gauche de la flèche.
- **Range** = la ou les classes que l'*objet* a le droit d'être. Le côté droit de la flèche.

Dans l'ontologie d'exemple, `LOCATED_IN` a pour domaine `{City, Country}` et pour range `{City, Person, Country}`. Lisez cela ainsi : « *la chose qui se situe quelque part peut être une City ou un Country ; la chose dans laquelle elle se situe peut être une City, une Person ou un Country.* »

Lister plusieurs classes dans un domain ou un range signifie que *n'importe laquelle d'entre elles* convient — c'est une union, pas une intersection.

C'est par le domain et le range que vous contraignez l'extraction. Une propriété au domain et au range étroits est un signal fort pour l'extracteur ; une propriété sans ni l'un ni l'autre correspondra à presque n'importe quoi. Commencez étroit, regardez le graphe extrait, et n'élargissez que là où vous constatez de vraies choses manquées.

Annotation

Une **annotation** est une métadonnée lisible par un humain, attachée à une classe, une propriété ou un individu. Elle n'a aucun effet sur la logique du graphe ; elle existe pour que vous, vos collègues et le modèle d'extraction puissiez dire ce qu'une chose signifie.

Les plus courantes :

- `rdfs:label` — le nom affiché. Perseus le renseigne pour vous quand vous nommez une ressource.
- `rdfs:comment` / `dcterms:description` / `skos:definition` — une description en prose.
- `rdfs:seeAlso` , `rdfs:isDefinedBy` — des pointeurs vers des ressources liées ou définissantes.

Les annotations sont **étiquetées par langue**. Chaque ligne d'annotation a un sélecteur de langue à droite (`en` (`English`) , ou le générique `lang` si rien n'est défini). C'est ainsi qu'une seule ontologie sert un pipeline multilingue : la même classe peut porter une étiquette anglaise et une étiquette française.

Les descriptions sont porteuses. L'extraction de Perseus s'appuie sur un LLM, et les annotations d'une classe font partie de ce que le modèle lit pour décider si un fragment de texte est une instance de cette classe. Un `rdfs:comment` sur `Person` disant « *an individual human named in the source text, not a fictional or hypothetical person* » change ce qui revient. Le laisser vide laisse le modèle deviner.

URI / IRI

Chaque ressource d'une ontologie a un identifiant unique au monde — une **URI** (ou sa forme internationalisée, une **IRI**) — affiché en gris en haut à droite de chaque panneau de détail :

```
http://www.w3.org/2002/07/owl#Country
http://example.org/ontology#employs
```

Cela *ressemble* à une URL, mais ce n'en est pas une, et la différence compte. Une URL est une *adresse* : elle indique où télécharger quelque chose, et elle n'offre aucune garantie d'unicité. Une URI est un *identifiant* : rien n'est téléchargé depuis là, et sa seule raison d'être est d'être unique au monde. C'est cette unicité qui permet de fusionner deux ontologies venant de deux équipes différentes sans que le `Person` de l'une entre en collision avec le `Person` de l'autre.

La partie après le # est le nom local, que Perseus dérive du nom que vous saisissez. Une ressource fraîchement créée et encore sans nom reçoit une URI provisoire contenant un horodatage (owl#timestamp-1783753869409) ; dès que vous tapez un vrai nom, l'URI se fixe à owl#Country . C'est le comportement attendu. Évitez simplement d'enregistrer une ressource tant qu'elle s'appelle encore *Untitled*.

Les préfixes que vous verrez dans les menus déroulants

Les préfixes `rdfs:` / `owl:` / `dcterms:` du menu déroulant des annotations sont des abréviations d'URIs longues. Savoir de quelle famille vient un terme aide à choisir le bon :

Préfixe	Signifie	À utiliser pour
<code>rdf:</code>	RDF core	le modèle brut de triplets ; <code>rdf:type</code>
<code>rdfs:</code>	RDF Schema	<code>rdfs:label</code> , <code>rdfs:comment</code> , sous-classe / sous-propriété
<code>owl:</code>	Web Ontology Language	classes, propriétés, <code>sameAs</code> , <code>differentFrom</code>
<code>skos:</code>	Simple Knowledge Organization System	<code>skos:definition</code> , vocabulaires de type thésaurus
<code>dc:</code> / <code>dcterms:</code>	Dublin Core	<code>title</code> , <code>description</code> , métadonnées documentaires génériques

Quand plusieurs d'entre eux proposent des termes quasi identiques (`rdfs:comment` contre `dcterms:description` contre `skos:definition`), choisissez-en un et tenez-vous-y dans toute l'ontologie. La cohérence compte plus que le choix lui-même.

Turtle (.ttl)

Dans Perseus, les ontologies sont stockées en fichiers **Turtle** — une sérialisation en texte brut de triplets RDF, d'où l'extension `.ttl` sur chaque ontologie de la bibliothèque. C'est lisible :

```
:San_Francisco  rdf:type      :City .
:San_Francisco  rdfs:label    "San Francisco"@en .
:San_Francisco  :LOCATED_IN  :USA .
```

Vous n'aurez jamais à écrire du Turtle pour utiliser l'éditeur. Mais tout ce que vous faites dans l'interface construit ces lignes, et si vous exportez une ontologie, c'est ce que vous obtenez.

Le Turtle est aussi votre arbitre. Si vous n'arrivez pas à dire si un champ a fait ce que vous vouliez — si cette ligne de Relationship a atterri comme une contrainte de schéma ou comme un vrai fait — enregistrez, puis ouvrez **View TTL** depuis le menu **...** de la carte de l'ontologie et lisez la source.

La place de WebProtégé

L'éditeur de Perseus est calqué sur **WebProtégé**, la version navigateur de Protégé, l'éditeur d'ontologies open source né à Stanford. Le modèle mental se transpose directement : la même séparation Classes / Properties / Individuals, la même sémantique domain/range, la même documentation par annotations, le même historique de révisions par commit accompagné d'un message.

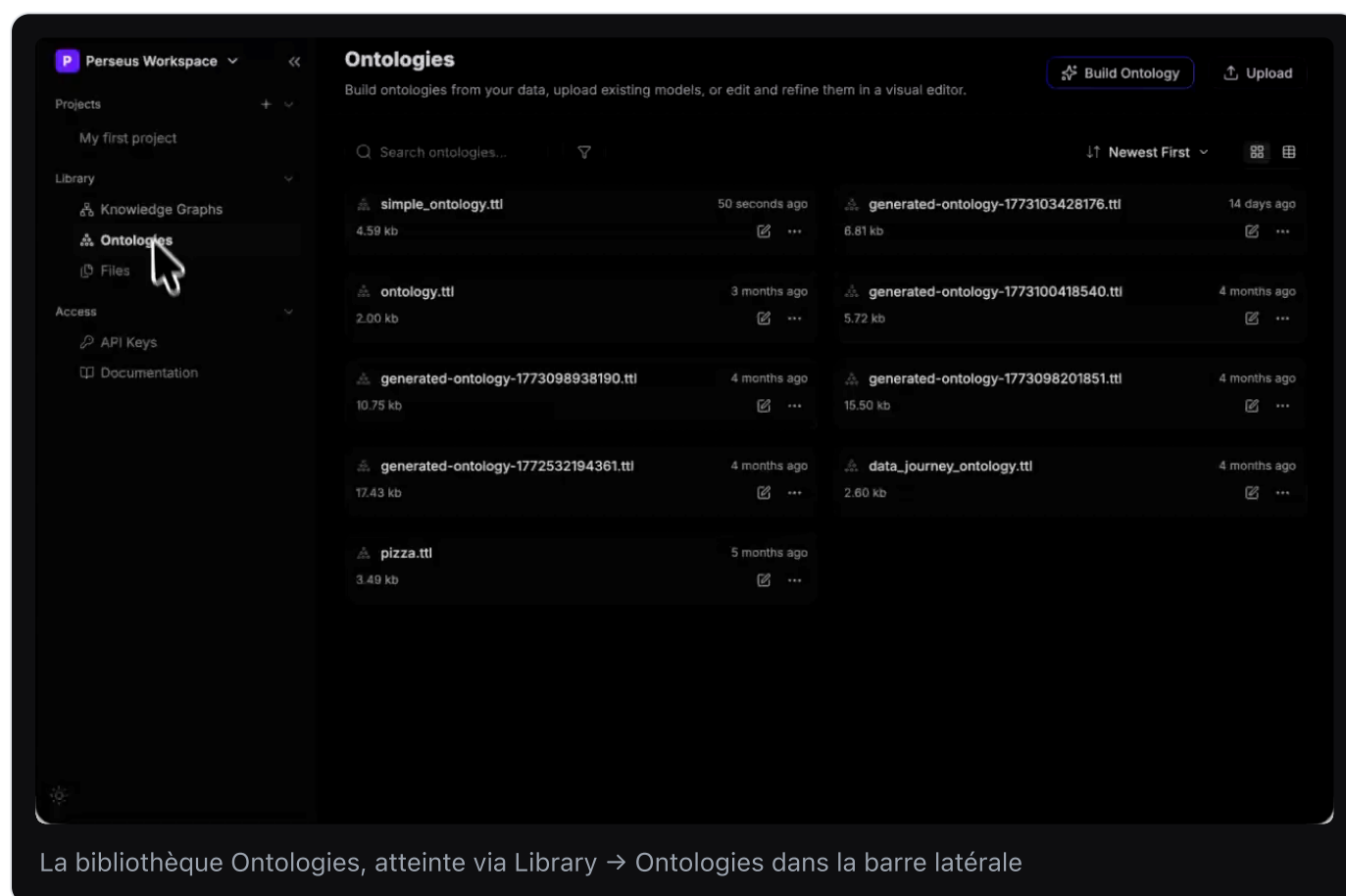
Perseus expose délibérément une surface plus petite. Il laisse de côté la machinerie OWL lourde de WebProtégé (expressions de classes complexes, restrictions, intégration d'un raisonneur, fils de discussion) au profit du sous-ensemble qui pilote l'extraction Perseus.

3. Accéder à l'éditeur

La bibliothèque Ontologies

Dans la barre latérale gauche de l'espace de travail Perseus, sous **Library**, cliquez sur **Ontologies**. (Les entrées voisines sont **Knowledge Graphs** — la sortie extraite — et **Files** — les documents sources.)

La page Ontologies liste chaque ontologie de l'espace de travail sous forme de carte affichant son nom de fichier, sa taille et sa date de dernière modification. Vous disposez d'un champ de recherche, d'un filtre, d'un tri **Newest First**, et d'un basculement entre vue grille et vue tableau.



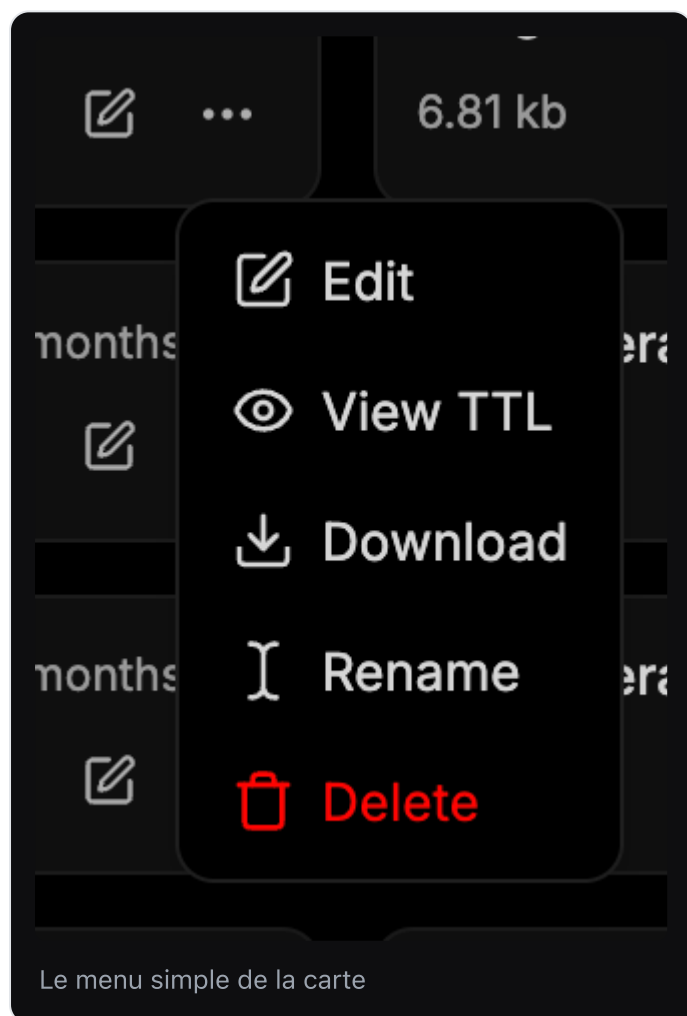
Chaque carte a une **icône crayon** (ouvrir dans l'éditeur) et un **menu ...**.

Le menu de la carte : simple et avancé

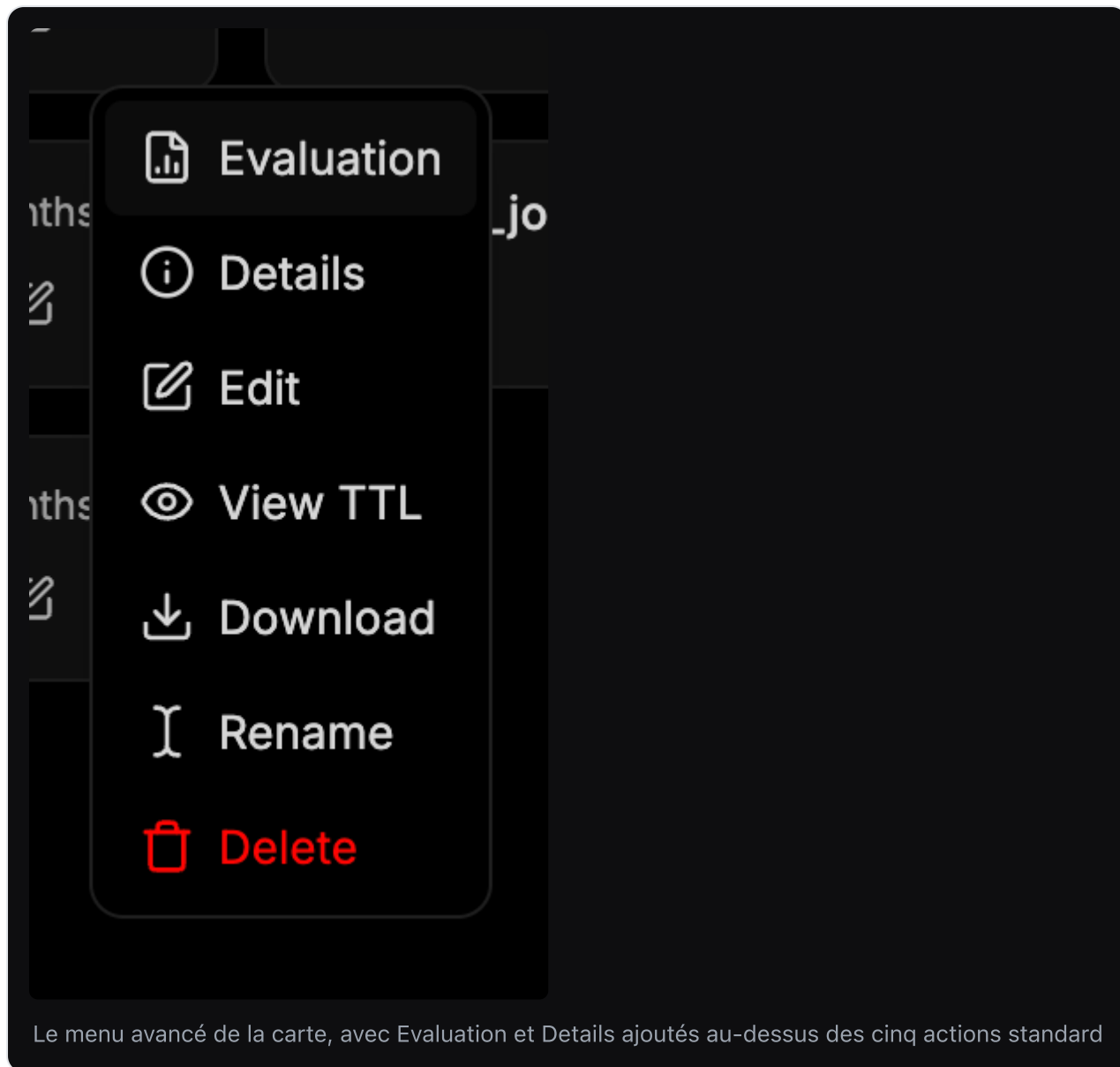
Le menu **...** existe en deux formes. Celle que vous obtenez dépend de l'ontologie.

Le **menu simple** propose cinq actions, et toutes les ontologies en disposent :

Action	Ce qu'elle fait
Edit	Ouvre l'ontologie dans l'éditeur visuel. Identique à l'icône crayon.
View TTL	Affiche la source Turtle brute — la vérité de terrain sur ce que l'éditeur a construit. Voir Turtle (.ttl).
Download	Exporte le fichier <code>.ttl</code> . Utilisez-le pour committer une ontologie dans votre propre dépôt, comparer deux versions hors de Perseus, ou la passer à un autre outil.
Rename	Renomme le fichier. Notez que cela renomme le <i>fichier</i> , pas les ressources qu'il contient.
Delete	Supprime l'ontologie de l'espace de travail.



Le **menu avancé** ajoute deux entrées en haut — **Evaluation** et **Details** :



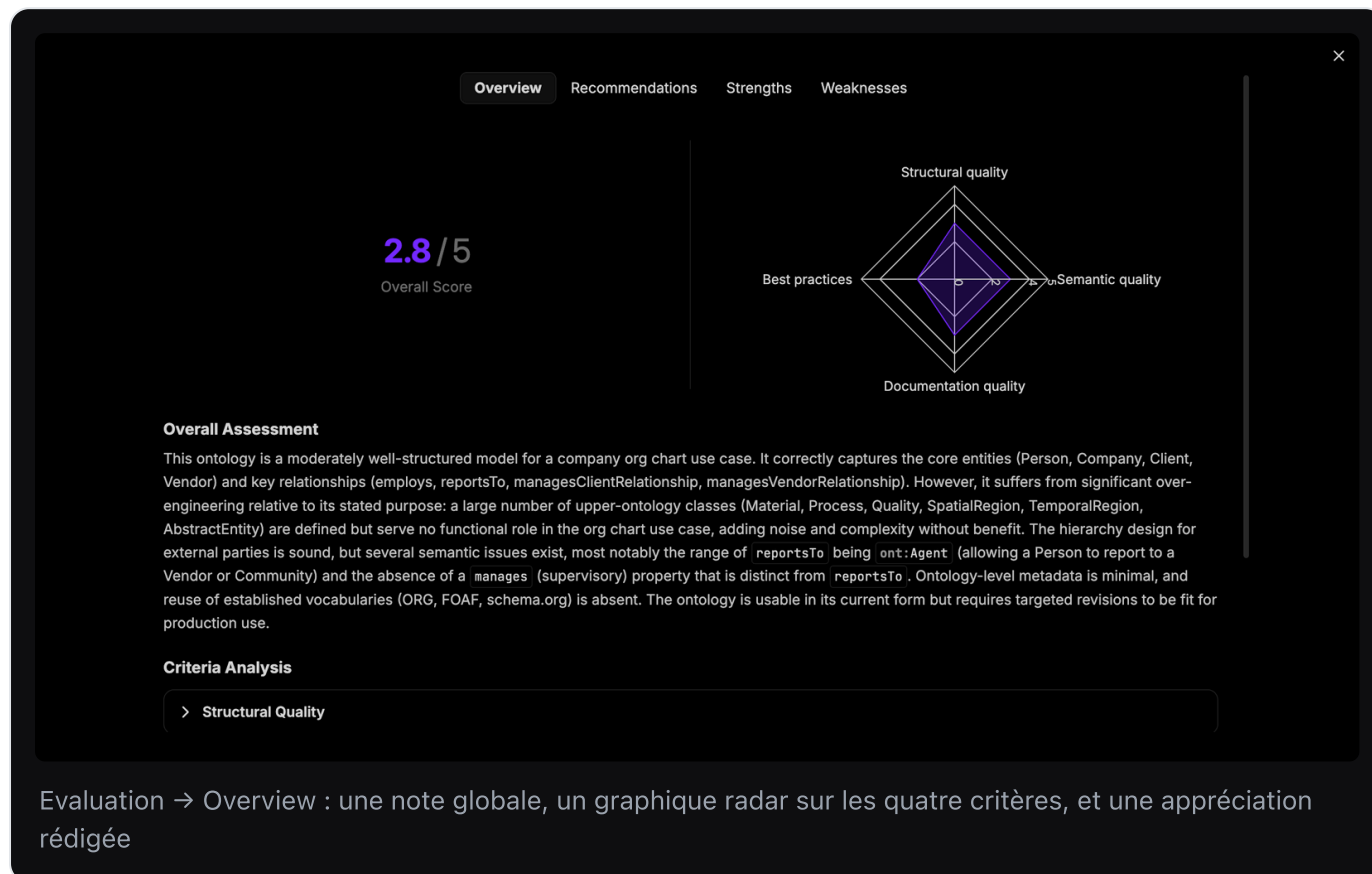
Ces deux suppléments sont des rapports sur la *façon dont l'ontologie a été produite*, pas sur son contenu actuel. Ils apparaissent sur les ontologies que Perseus a générées avec **Build Ontology**, parce que cette génération s'exécute comme un job et que c'est le job qui les produit : la chronologie, le cas d'usage que vous avez saisi, et l'analyse de qualité. Une ontologie arrivée par un autre chemin — téléversée, ou convertie depuis un autre schéma — n'a aucun job derrière elle, et n'obtient donc que le menu simple.

Deux conséquences à connaître :

- Modifier et enregistrer une ontologie ne rafraîchit **pas** son Evaluation. Le rapport décrit l'ontologie telle que la génération l'a produite, il devient donc obsolète dès que vous commencez à l'éditer.
- Si vous téléchargez une ontologie générée puis la téléversez à nouveau, la copie n'affiche que le menu simple, alors même que le fichier est identique octet pour octet. Les rapports appartiennent au job de génération, pas au fichier.

Evaluation

Une analyse de la qualité de l'ontologie générée, notée sur 5 selon quatre critères — **Structural quality** (qualité structurelle), **Semantic quality** (qualité sémantique), **Documentation quality** (qualité de la documentation) et **Best practices** (bonnes pratiques) — présentée sous forme de graphique radar avec, en dessous, une appréciation générale en prose.



Quatre sous-onglets :

- **Overview** — la note globale, le graphique radar, l'appréciation rédigée, et une section **Criteria Analysis** (analyse par critère) que vous pouvez déplier critère par critère.
- **Recommendations** — des changements concrets et actionnables, chacun accompagné d'une justification. Ils sont précis : « *Restreindre le range de `ont:reportsTo` de `ont:Agent` à `ont:Person`* », « *Supprimer les classes `ont:Material`, `ont:Process` ... ainsi que leurs axiomes de disjonction.* » C'est l'onglet le plus directement utile — c'est une liste de travail que vous pouvez emporter telle quelle dans l'éditeur.
- **Strengths** — ce que le générateur a réussi, et pourquoi c'est juste.
- **Weaknesses** — là où le modèle est faux ou surdimensionné, avec le raisonnement.

OverviewRecommendationsStrengthsWeaknesses

Add a named inverse property for `ont:reportsTo`, such as `ont:directlyManages`, with domain `ont:Person` and range `ont:Person`, and declare it as `owl:inverseOf ont:reportsTo`.

The primary use case is to 'show who works for who and who manages which clients and vendors.' Without a named supervisory property, the managerial hierarchy direction is not directly queryable, which is a fundamental gap for an org chart.

Restrict the range of `ont:reportsTo` **from** `ont:Agent` **to** `ont:Person` **(or a new class** `ont:Employee` **that is a subclass of** `ont:Person` **employed by the company).**

Allowing a Person to report to a Vendor, Community, or any Agent is semantically incorrect for an org chart. This range is too broad and would allow invalid instance data to pass without detection, undermining data integrity.

Remove or move to a separate 'upper ontology' module the classes `ont:Material`, `ont:Process`, `ont:Quality`, `ont:SpatialRegion`, `ont:TemporalRegion`, and `ont:AbstractEntity`, **along with all their associated disjointness axioms.**

These classes serve no role in the org chart use case and constitute the majority of the ontology's axiom count. Their presence obscures the core model and adds maintenance burden without any benefit to the stated purpose.

Fix the domain of `ont:hasIndustry` **from** `ont:Company` **to** `ont:Organization` **to prevent the reasoner from incorrectly inferring that any organization with an industry attribute is a Company, which would violate the Company/CommercialPartner disjointness axiom.**

As currently defined, asserting `vendorX ont:hasIndustry 'Technology'` causes a reasoner to infer `vendorX` is a `Company`, which is disjoint with `CommercialPartner` (`vendorX`'s actual class), making `vendorX`'s class membership unsatisfiable.

Evaluation → Recommendations : chaque recommandation nomme la ressource exacte à changer et explique pourquoi

OverviewRecommendationsStrengthsWeaknesses

The Client/Vendor hierarchy under `CommercialPartner` is logically sound and correctly uses `owl:disjointWith` to prevent a single organization from being simultaneously a Client and a Vendor, which is a meaningful business constraint for the use case.

The use of `rdfs:subPropertyOf` to relate `ont:managesClientRelationship` and `ont:managesVendorRelationship` to the superproperty `ont:managesExternalRelationship` is a correct and useful design pattern, enabling general queries over all external relationship management.

Explicit inverse properties (`ont:employs` / `ont:isEmployedBy`) are correctly declared with `owl:inverseOf`, enabling bidirectional traversal of the employment relationship without redundant data entry.

The `ont:reportsTo` property is correctly annotated as `owl:AsymmetricProperty` and `owl:IrreflexiveProperty`, which are semantically appropriate constraints that prevent a person from reporting to themselves and prevent mutual reporting relationships.

All classes and properties include both `rdfs:label` and `rdfs:comment` annotations in English, providing a baseline level of human-readable documentation for every term.

Evaluation → Strengths

OverviewRecommendationsStrengthsWeaknesses

Severe over-engineering relative to the use case

Classes `ont:Material`, `ont:Process`, `ont:Quality`, `ont:SpatialRegion`, `ont:TemporalRegion`, and `ont:AbstractEntity` are defined with full disjointness axioms but have no properties, no subclasses relevant to the org chart, and no role in the stated purpose of showing who works for whom and who manages clients/vendors. This inflates the ontology with dead weight.

The range of `ont:reportsTo` is `ont:Agent`, which is far too broad. This allows a Person to report to a Vendor, a Community, or an ExternalCommunity, which is semantically incorrect for an org chart. The range should be restricted to `ont:Person` or at most `ont:Employee` / `ont:Manager`.

There is no explicit 'manages' or 'isManagedBy' supervisory property between Persons. The org chart use case requires modeling who manages whom (the managerial hierarchy), but the ontology only provides `ont:reportsTo` (the subordinate's perspective) without a named inverse property, making it harder to query 'who does Person X manage?'

The `ont:hasIndustry` datatype property has its domain set to `ont:Company`, but it would logically apply to any `ont:Organization` (e.g., a Vendor or ResearchInstitution also has an industry). This domain restriction is unnecessarily narrow and inconsistent with the broader class hierarchy.

No ontology-level metadata beyond a single `rdfs:comment` is provided. There is no `owl:versionInfo`, `dcterms:creator`, `dcterms:created`, `dcterms:title`, or `owl:imports` declaration, making the ontology difficult to manage, version, and discover.

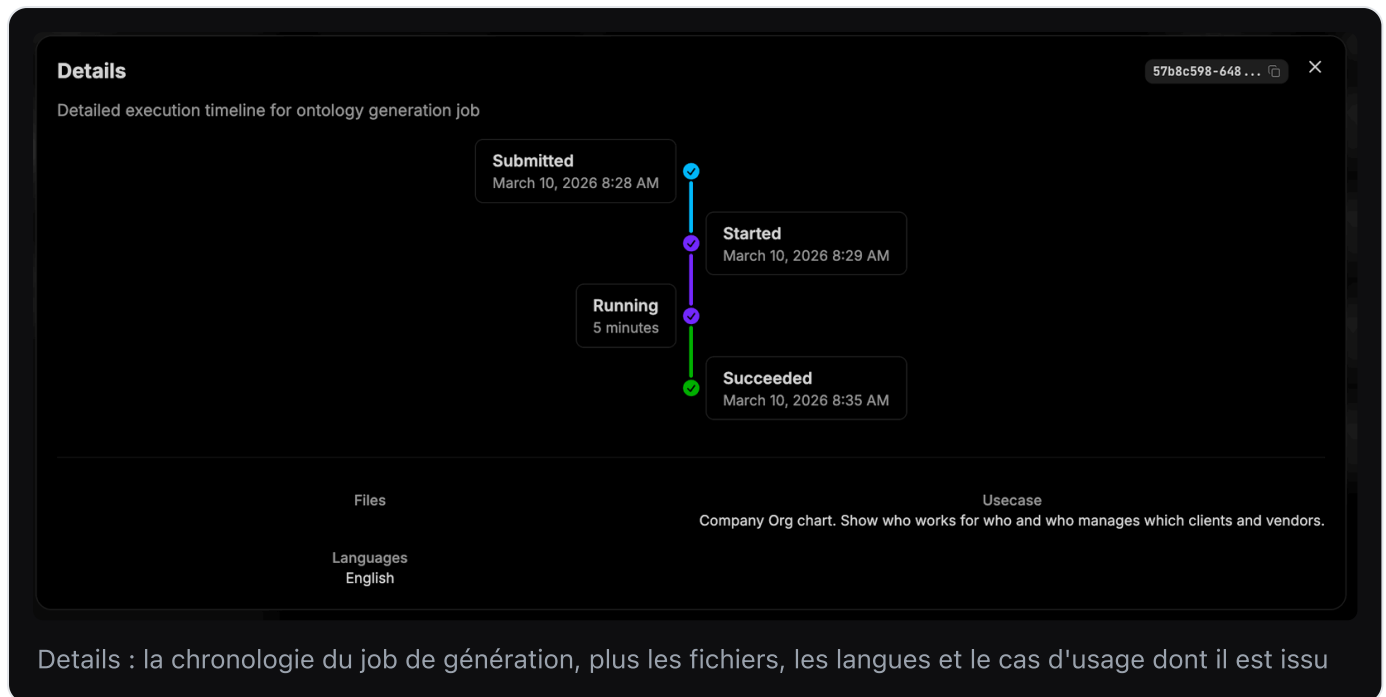
The ontology does not reuse any established vocabularies. The W3C Organization Ontology (ORG), FOAF, or schema.org already define Person, Organization, membership, and reporting relationships with broad community adoption. Reinventing these concepts creates interoperability barriers.

Evaluation → Weaknesses

L'évaluation juge l'ontologie *au regard du cas d'usage que vous avez décrit*, et c'est pour cela que ce cas d'usage mérite d'être rédigé avec soin — c'est l'étalon. Un constat fréquent est la sur-ingénierie : le générateur émet des classes d'ontologie supérieure (`Material`, `Process`, `Quality`, `SpatialRegion`, `TemporalRegion`, `AbstractEntity`) qui ne jouent aucun rôle dans votre cas d'usage réel et ne font qu'ajouter du bruit.

Details

Le registre d'exécution du job de génération : une chronologie **Submitted** → **Started** → **Running** → **Succeeded** (soumis → démarré → en cours → réussi) avec horodatages et durée, l'identifiant du job, les **Files** à partir desquels il a été construit, les **Languages** sélectionnées, et le texte de l'**Usecase** que vous avez saisi dans l'assistant.



C'est ici qu'il faut regarder quand vous avez oublié à *quoi* servait une ontologie générée — le cas d'usage y est consigné mot pour mot, et nulle part dans l'éditeur.

Trois façons d'obtenir une ontologie

1. **Build Ontology** — laissez Perseus en générer une à partir de documents que vous avez déjà téléversés. Commencez par là. Générer à partir d'une poignée de documents représentatifs vous emmène l'essentiel du chemin, et ce qui reste est bien plus facile à voir avec quelque chose de concret devant vous qu'à partir d'une ontologie vide.
2. **Upload** — apportez un fichier `.ttl` existant. Utile si vous maintenez déjà une ontologie ailleurs, ou si vous importez un vocabulaire standard.
3. **Ouvrez-en une existante** et modifiez-la. Les ontologies générées par Perseus s'appellent `generated-ontology-<timestamp>.ttl` jusqu'à ce que vous les renommiez.

Construire une ontologie à partir de vos fichiers

Build Ontology ouvre un assistant en trois étapes :

1. Select files — cochez les fichiers de l'espace de travail dont dériver l'ontologie. Perseus les lit pour déterminer quelles classes et quelles relations votre domaine contient, alors choisissez des fichiers représentatifs plutôt qu'exhaustifs.

Build ontology

Select files

Describe usecase

Select languages

Select files

Select files from your workspace to build an ontology from.

Q Search files...

☐ Org_part2.md1 month ago

☐ Org_part1.md1 month ago

☐ Events.md1 month ago

☐ People.md1 month ago

☐ Org.md1 month ago

☐ Hackersquad.md1 month ago

☐ 2024-06-11 HackNight.md1 month ago

☐ Adam Chan.md1 month ago

→ Configure usecase

Étape 1 — Select files

2. Describe usecase — un champ de texte libre décrivant ce que vous modélisez et à quoi va servir le graphe. Cela oriente ce que le générateur juge assez important pour devenir une classe, alors soyez précis sur les questions auxquelles le graphe fini doit répondre.

Build ontology

Select files

Describe usecase

Select languages

Describe usecase

Provide a description of the use case for the ontology.
See the [documentation](#) on ontology building and filling the use case field.

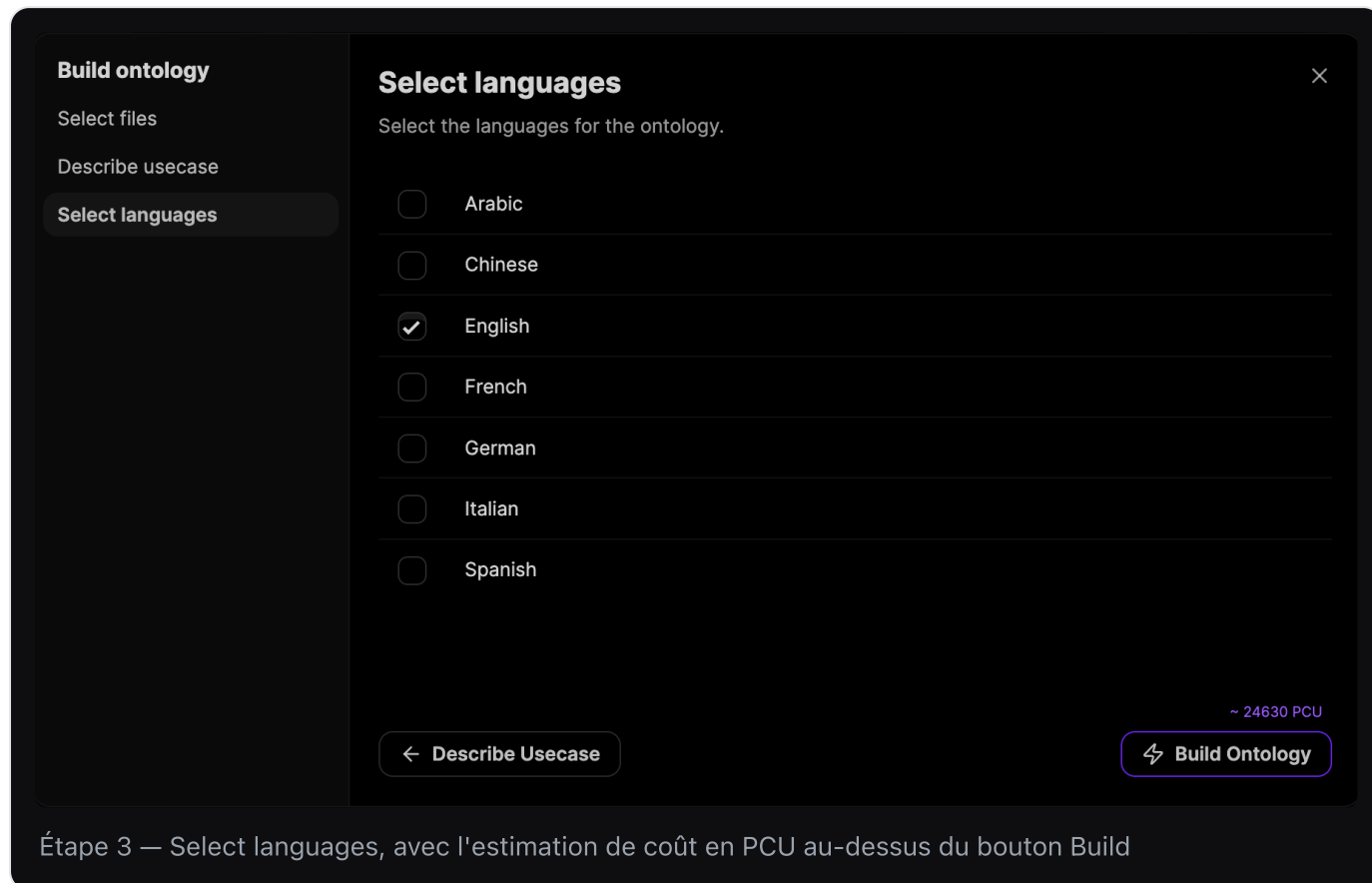
Describe your usecase

← Select Files

→ Select Languages

Étape 2 — Describe usecase

3. Select languages — cochez les langues dans lesquelles l'ontologie doit porter des étiquettes (arabe, chinois, anglais, français, allemand, italien, espagnol). Cela pilote les étiquettes de langue sur les annotations générées.



The screenshot shows a dark-themed interface for building an ontology. On the left, a sidebar titled 'Build ontology' contains three buttons: 'Select files', 'Describe usecase', and 'Select languages' (which is highlighted). The main area is a modal dialog titled 'Select languages' with a close button (X) in the top right. Inside the dialog, the instruction 'Select the languages for the ontology.' is followed by a list of languages with checkboxes: Arabic, Chinese, English (checked), French, German, Italian, and Spanish. At the bottom right of the dialog, the estimated cost is shown as '~ 24630 PCU'. Two buttons are at the bottom: '← Describe Usecase' and '⚡ Build Ontology'.

Build ontology

Select files

Describe usecase

Select languages

Select languages

Select the languages for the ontology.

☐ Arabic

☐ Chinese

☒ English

☐ French

☐ German

☐ Italian

☐ Spanish

~ 24630 PCU

← Describe Usecase

⚡ Build Ontology

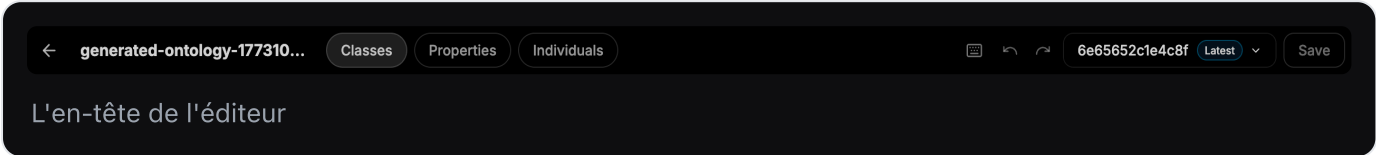
Étape 3 — Select languages, avec l'estimation de coût en PCU au-dessus du bouton Build

Le dernier écran affiche un coût estimé en **PCU** (Perseus Compute Units) avant que vous ne vous engagiez. Cliquez sur **Build Ontology** pour lancer le traitement. À la fin, la nouvelle ontologie apparaît dans la bibliothèque.

Attendez-vous à devoir la retoucher. Une ontologie générée est un bon premier jet, pas un objet fini. Avant d'ouvrir l'éditeur, ouvrez le menu **...** de la carte → **Evaluation** → **Recommendations** : la génération a déjà relu sa propre sortie et vous dit quoi corriger. Traiter cette liste est le chemin le plus rapide vers une ontologie utilisable.

4. La disposition de l'éditeur

Ouvrez une ontologie et vous obtenez un éditeur plein écran.

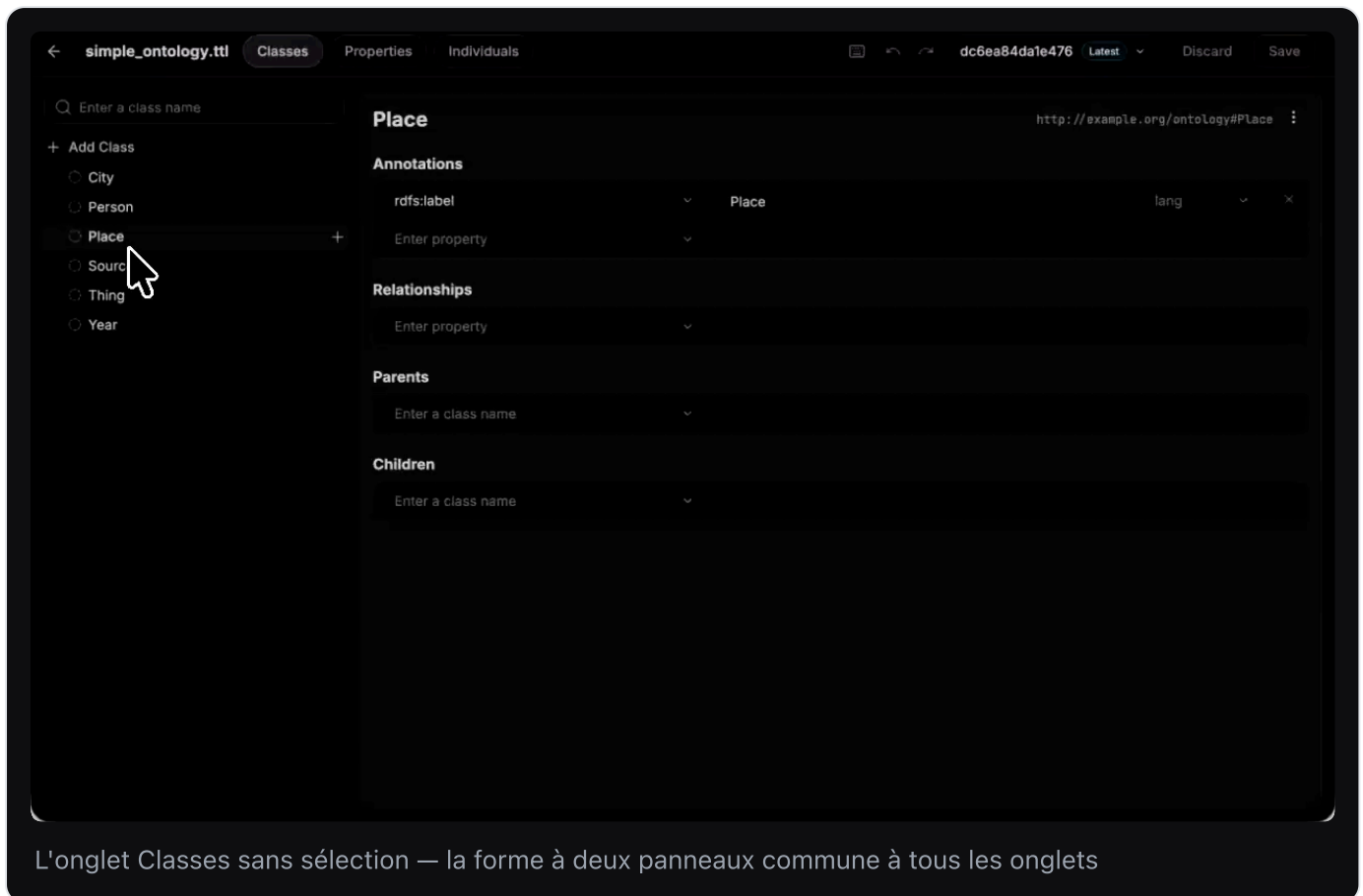


L'en-tête, de gauche à droite :

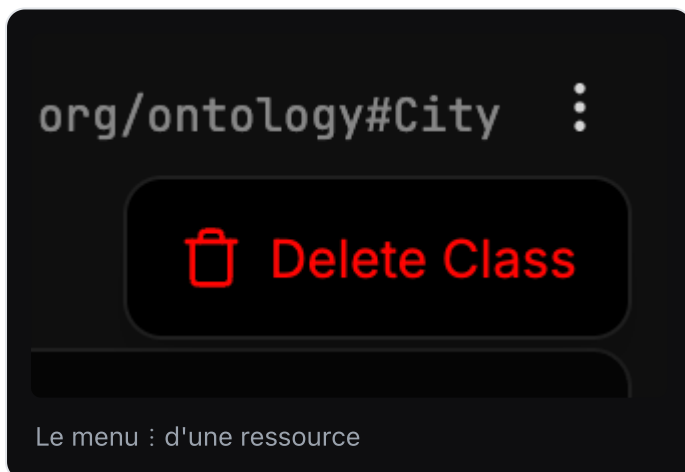
Élément	Ce qu'il fait
←	Retour à la bibliothèque Ontologies.
Nom de fichier	Le nom de l'ontologie que vous avez ouverte.
Classes / Properties / Individuals	Les trois onglets. Chacun est une vue différente de la même ontologie.
Icône ≡	Ouvre la fenêtre des raccourcis clavier.
↶ / ↷	Annuler / rétablir.
Pastille de version	Le hash du commit courant (par ex. <code>cd494f51312665</code>) avec un badge Latest . Cliquez sur le chevron pour dérouler l'historique des versions.
Discard	Jette toutes les modifications non enregistrées.
Save	Committe vos modifications avec un message.

Sous l'en-tête, chaque onglet adopte la même forme à deux panneaux :

- **Panneau de gauche** — une liste cherchable et filtrable des ressources de ce type, plus un bouton **Add** en haut.
- **Panneau de droite** — le formulaire de détail de ce que vous avez sélectionné. Vide tant que vous n'avez rien sélectionné (« No class selected — Select a class in the left panel to inspect it »).



Chaque panneau de détail affiche le **nom de la ressource comme un titre éditable**, son **URI** en gris en haut à droite, et un **menu** à côté de l'URI.



Ce menu ne contient qu'une seule action — **Delete Class** (ou Delete Property / Delete Individual, selon l'onglet). C'est le seul moyen de supprimer une ressource, à part le raccourci .

Supprimer n'est pas une opération douce : si d'autres ressources référençaient celle qui disparaît — une classe utilisée comme domain d'une propriété, un individu à l'autre bout d'une relation — ces références partent avec elle. La suppression n'est validée qu'au moment du **Save**, donc permet de revenir en arrière et **Discard** annule tout d'un bloc.

Les modifications sont gardées en mémoire pendant que vous travaillez. Rien n'est persisté tant que vous n'appuyez pas sur **Save**.

5. Travailler avec les classes

L'onglet **Classes** est l'endroit où vous définissez les types de choses de votre domaine.

Ajouter une classe

Deux façons :

- **+ Add Class** en haut du panneau de gauche — crée une nouvelle classe de premier niveau.
- Survolez une classe existante et cliquez sur le **+** qui apparaît à sa droite — crée une nouvelle classe *comme enfant de celle-ci*. C'est la façon rapide de construire une hiérarchie.

Dans les deux cas, vous obtenez une classe nommée **Untitled**, sélectionnée, avec une URI provisoire à horodatage. Écrivez par-dessus le titre dans le panneau de droite pour la nommer. Perseus inscrit le nom dans `rdfs:label` et réécrit l'URI en conséquence.

Convention de nommage : singulier, PascalCase — Country, pas countries.

Le panneau de détail de la classe

Sélectionner une classe fait apparaître quatre sections.

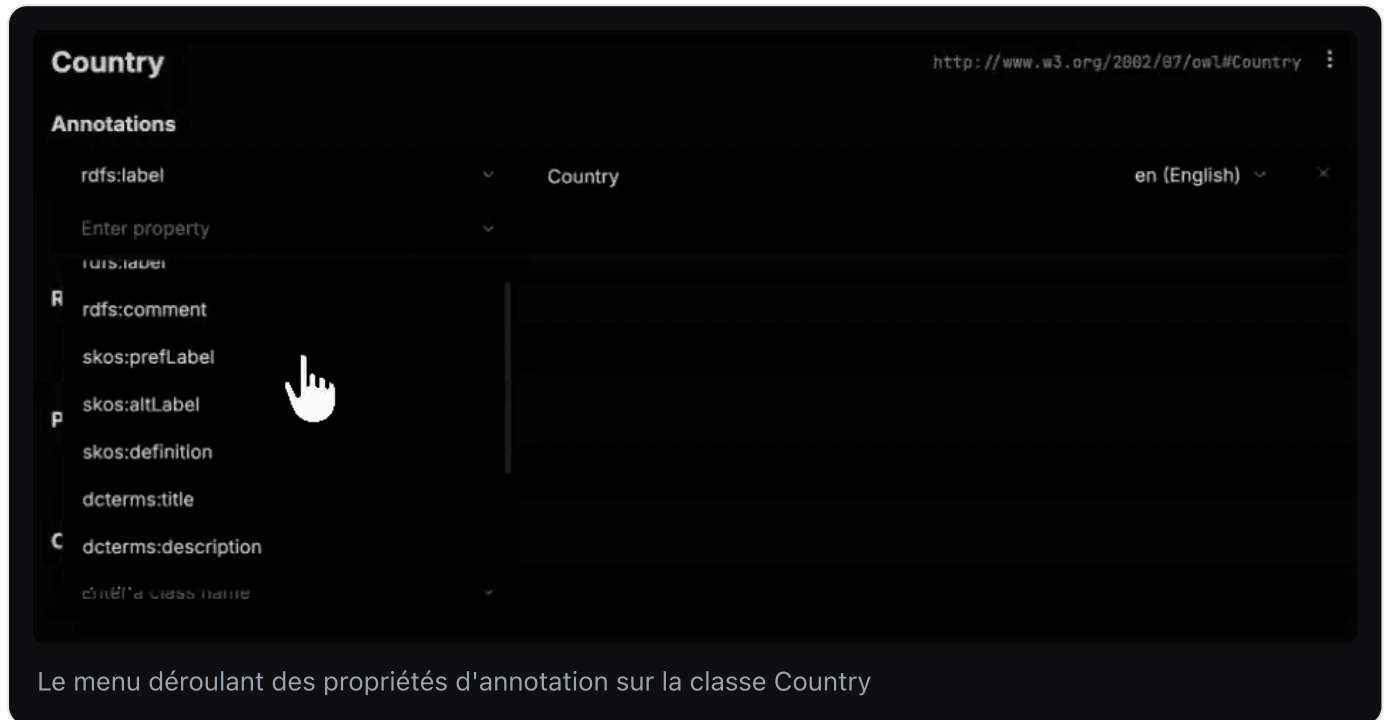
The screenshot shows the Perseus ontology editor interface. The top bar displays the ontology name 'generated-ontology-177310...' and tabs for 'Classes', 'Properties', and 'Individuals'. The 'Classes' tab is active. On the left, a sidebar lists class types: 'Add Class', 'Abstract Entity', 'Agent' (selected), 'Material', 'Process', 'Quality', 'Spatial Region', 'Temporal Region', and 'test'. The main panel shows the details for the 'Agent' class. It includes a search bar 'Enter a class name', the class name 'Agent', and its URI 'http://example.org/ontology#Agent'. The 'Annotations' section shows two entries: 'rdfs:label' with the value 'Agent' and 'rdfs:comment' with the value 'Entities capable of intentional action and decision-making, including persons and organizations.' The 'Relationships' section has a field 'Enter property'. The 'Parents' section has a field 'Enter a class name'. The 'Children' section lists two classes: 'Organization' and 'Person'.

Le panneau de détail de la classe. Agent a deux annotations, aucune relation, aucun parent, et deux enfants

Annotations

De la documentation en forme libre. Chaque ligne est : **propriété + valeur + étiquette de langue**.

Cliquez sur le menu déroulant **Enter property** pour choisir le type d'annotation. La liste déroule `rdfs:label`, `rdfs:comment`, `skos:prefLabel`, `skos:altLabel`, `skos:definition`, `dcterms:title`, `dcterms:description`, `dc:title`, `dc:description`, `rdfs:seeAlso` et `rdfs:isDefinedBy`. Saisissez la valeur à droite, réglez la langue tout à droite, et ajoutez une autre ligne au besoin. Le `×` supprime une ligne.



Country http://www.w3.org/2002/07/owl#Country

Annotations

rdfs:label	Country	en (English) ×
Enter property		
rdfs:label		
rdfs:comment		
skos:prefLabel		
skos:altLabel		
skos:definition		
dcterms:title		
dcterms:description		
Enter a class name		

Le menu déroulant des propriétés d'annotation sur la classe Country

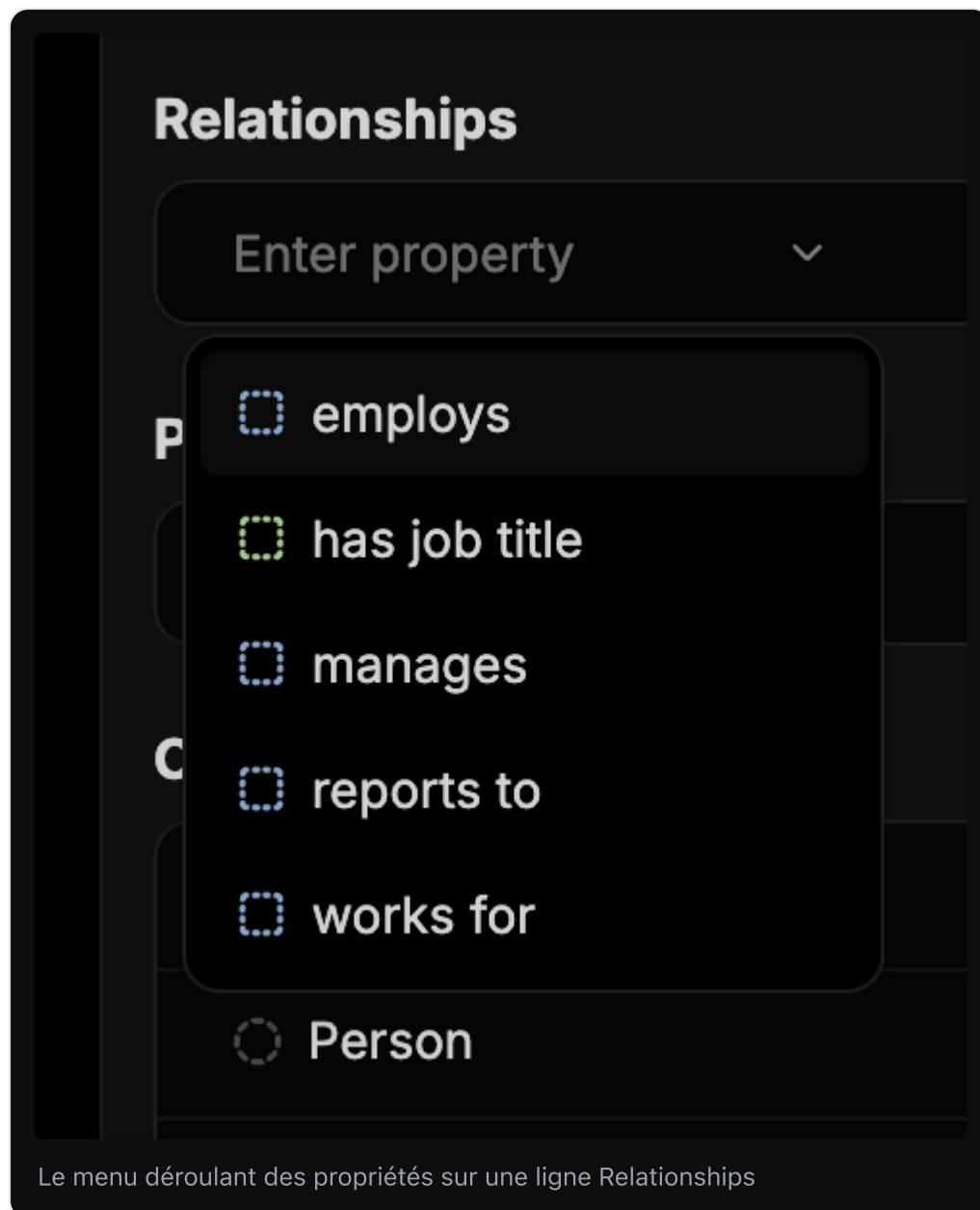
Une classe fraîchement nommée arrive avec `rdfs:label` déjà rempli. Ajoutez-y aussi une description — elle fait partie du prompt que lit l'extracteur, donc elle affecte directement la qualité de l'extraction. Voir Annotation.

`skos:prefLabel` et `skos:altLabel` valent le détour : le premier est le nom d'affichage préféré, le second héberge les synonymes. Poser des alias connus sur une classe (`Firm`, `Corporation` comme altLabels de `Company`) donne à l'extracteur plus de prise pour trouver des correspondances.

Relationships

Les relations qui peuvent *partir* de cette classe. Chaque ligne comporte **deux menus déroulants** : la **propriété**, puis la **classe à l'autre bout de celle-ci**.

Le menu de la propriété liste toutes les propriétés d'objet définies dans l'ontologie (`ASSOCIATED_WITH`, `description`, `LOCATED_IN`, `MENTION`, `name`, `url` dans l'ontologie d'exemple).



Une fois la propriété choisie, le second menu — **Enter a class name** — propose toutes les classes de l'ontologie. C'est la classe située de l'autre côté de la relation.

Après avoir choisi has delegated authority from, le second menu propose la classe cible

Lisez une ligne complète comme une phrase. `ManagingGeneralAgent` + `has delegated authority from` + `InsuranceCarrier` dit : « *un managing general agent peut détenir une délégation d'autorité d'un assureur.* »

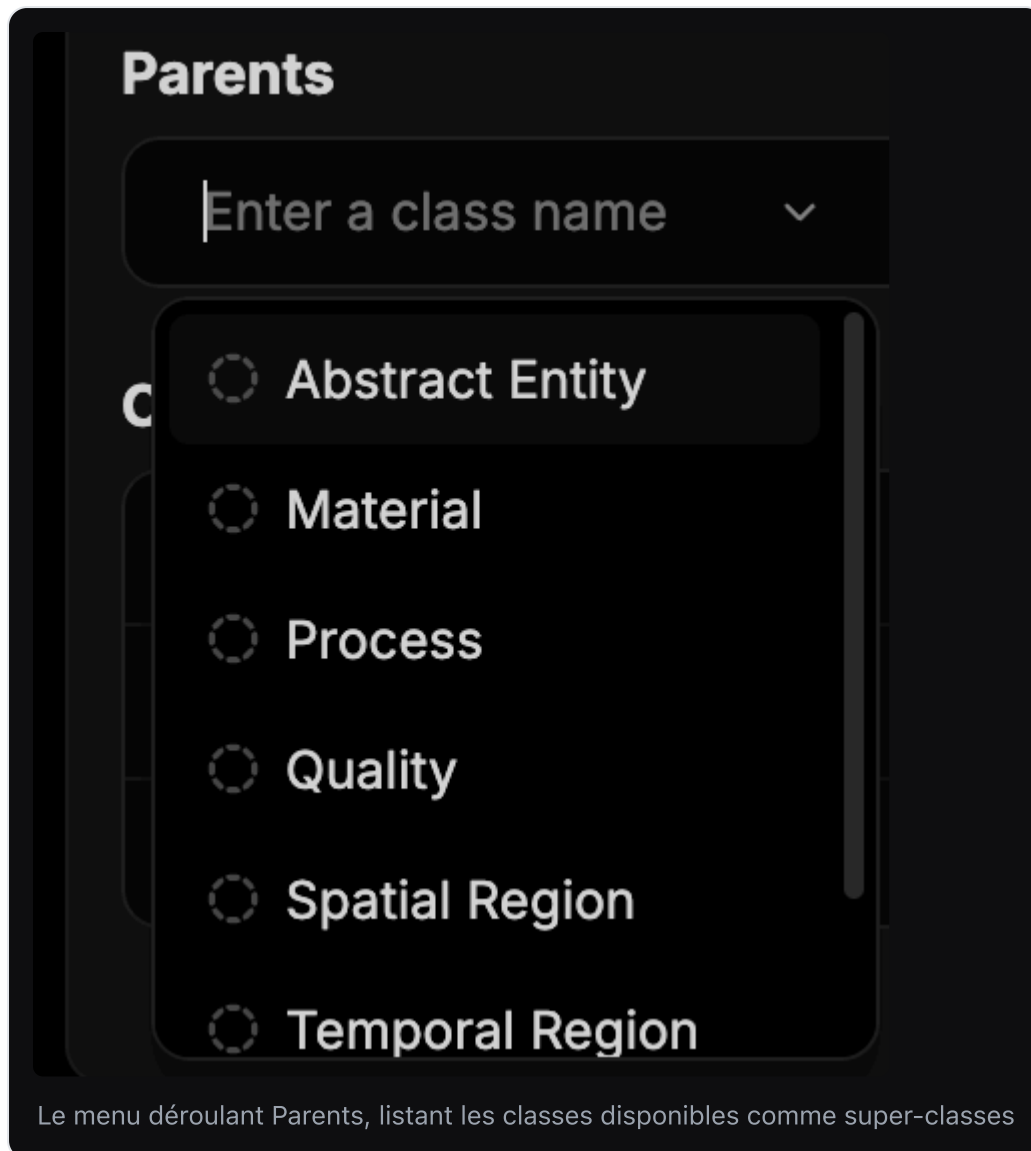
C'est exactement le même fait que le **Domain** et le **Range** de la propriété, saisi depuis l'autre côté : la classe que vous consultez entre dans le Domain de la propriété, et la classe que vous choisissez à droite entre dans son Range. Vous pouvez travailler des deux bouts — la boîte Relationships de la classe est le côté pratique quand vous raisonnez classe d'abord, les boîtes Domain et Range de la propriété le sont quand vous raisonnez relation d'abord. Voir Le panneau de détail de la propriété.

L'onglet Properties fait foi. Si vous avez un doute sur ce qu'une ligne Relationships a réellement affirmé, ouvrez la propriété là-bas et lisez son Domain et son Range.

Parents et Children

La hiérarchie des sous-classes.

- **Parents** — les classes dont celle-ci est une sous-classe. Tapez ou choisissez un nom de classe. Une classe peut avoir plusieurs parents.
- **Children** — les classes qui sont des sous-classes de celle-ci.



Les deux sont deux vues du même fait. Donner `Place` comme parent de `City` est identique à ajouter `City` aux children de `Place` ; l'autre panneau se met à jour tout seul. Prenez le sens dans lequel vous êtes en train de penser.

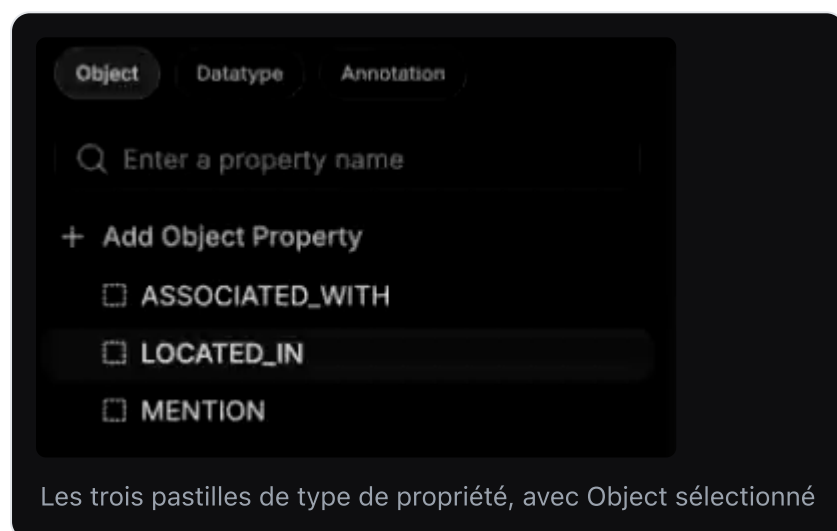
Parent/enfant veut dire est-un strict, pas « lié à » : tout ce qui est vrai du parent est vrai des enfants. Si vous ne diriez pas « tout X est un Y », ne faites pas de X un enfant de Y — modélisez-le plutôt comme une propriété d'objet entre les deux. Traiter le est-un comme un « lié à » polyvalent est l'erreur de modélisation la plus fréquente.

6. Travailler avec les propriétés

L'onglet **Properties** est l'endroit où vous définissez les arêtes.

Les trois onglets de propriétés

Trois pastilles de filtre en haut du panneau de gauche — **Object**, **Datatype**, **Annotation** — changent le type de propriété que vous regardez. Le bouton Add se renomme en conséquence (+ Add Object Property , + Add Datatype Property , ...).



Object = entité vers entité, datatype = entité vers littéral, annotation = documentation. Voir Propriété pour choisir entre elles.

Le panneau de détail de la propriété

Sélectionnez une propriété et vous obtenez :

The screenshot shows a web application for editing an ontology. At the top, there's a navigation bar with tabs for 'Classes', 'Properties', and 'Individuals'. The 'Properties' tab is active. On the left, there's a sidebar with a search bar and a list of properties: 'employs', 'manages', 'reports to', and 'works for'. The main area displays the details for the 'employs' property. It has a URL 'http://example.org/ontology#employs'. Under 'Annotations', there are two entries: 'rdfs:label' with the value 'employs' and 'rdfs:comment' with the value 'Indicates that a company has an employment or affiliation relationship with a person. Inverse of worksFor.'. Below this, there are sections for 'Domain' (with 'Company' selected) and 'Range' (with 'Person' selected). At the bottom, there's a 'Parents' section. A footer note reads: 'Lisez cet exemple comme 'une Company employs une Person' — une entreprise emploie une personne'.

- **Annotations** — le même widget que sur les classes. `rdfs:label` plus, idéalement, un `rdfs:comment` expliquant exactement quand cette relation s'applique. Dans les données d'exemple, `employs` porte le commentaire « *Indicates that a company has an employment or affiliation relationship with a person. Inverse of worksFor.* » — c'est le standard à viser.
- **Domain** — les classes autorisées à *gauche* de cette relation. Ajoutez avec **Add a domain class...**, retirez avec le `×`.
- **Range** — les classes autorisées à *droite*. Ajoutez avec **Add a range class...**.
- **Parents / Children** — la hiérarchie des sous-propriétés. Rarement nécessaire, mais cela vous permet de dire par exemple que `manages` est une spécialisation de `works with` ; chaque triplet `manages` compte alors aussi comme un triplet `works with`.

Domain et range sont les champs qui font le vrai travail — bien les régler, c'est l'essentiel de ce qui rend l'extraction précise. Voir Domaine et co-domaine.

LOCATED_IN

http://example.org/ontology#locatedIn

Annotations

rdfs:label

LOCATED_IN

lang

Enter property

Domain

☐ City

☐ Country

Add a domain class...

Range

☐ City

☐ Person

Add a range class...

Parents

Enter a property name

Children

Enter a property name

Ajouter une propriété

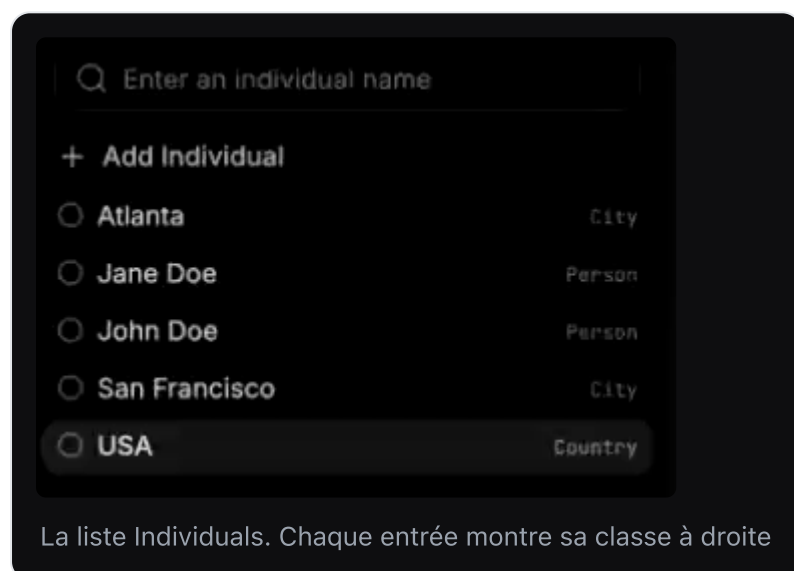
Cliquez sur `+ Add Object Property` (ou son équivalent datatype/annotation), nommez-la dans le titre, puis réglez son domain et son range. Une propriété au domain et au range vides est légale mais non contrainte, et correspondra bien plus largement que vous ne le voulez. Commencez étroit, élargissez ensuite.

Convention de nommage : les ontologies générées utilisent deux styles — `SCREAMING_SNAKE_CASE` pour les propriétés d'objet (`LOCATED_IN` , `ASSOCIATED_WITH`) et `lowerCamelCase` ou des mots simples pour les propriétés de type de données (`name` , `url` , `has job title`). Ni l'un ni l'autre n'est obligatoire. Choisissez une convention par ontologie et tenez-la.

7. Travailler avec les individus

L'onglet **Individuals** contient les entités concrètes et nommées. Le panneau de gauche liste chaque individu avec sa classe en gris à droite (`San Francisco` – `City`).

Les individus que vous déclarez ici vivent dans l'*ontologie*, pas dans le graphe extrait. Déclarez-les pour les raisons vues sous **Individu** : comme valeurs d'énumération fixes, comme entités de référence pour réconcilier les données extraites, ou comme fixtures de test pour un raisonneur. Les choses concrètes que l'extraction tire de vos documents sont des *instances* dans le graphe de connaissances — une couche distincte que cet éditeur ne montre pas.



Ajouter un individu

Cliquez sur `+ Add Individual` , puis tapez son nom par-dessus le titre. Comme pour les classes, il démarre en ressource sans nom avec une URI à horodatage, qui se fixe une fois que vous le nommez.

Un individu sans type ne sert à rien, donc l'étape suivante est toujours Types.

Types

La section **Types** est l'endroit où vous déclarez de quelle(s) classe(s) cet individu est une instance. Choisissez dans le menu déroulant des classes. Une fois réglée, la classe apparaît à côté du nom de l'individu dans le panneau de gauche.

C'est le triplet `rdf:type` — `:San_Francisco` `rdf:type` `:City` . Un individu peut avoir plusieurs types.

Relations entre individus

La section **Relationships** est l'endroit où vous affirmez des faits réels. Notez en quoi elle diffère de la section homonyme sur une *classe* :

- Sur une **classe**, Relationships déclare ce qui est *possible* (le schéma).
- Sur un **individu**, Relationships énonce ce qui est *vrai* (les données).

Chaque ligne est faite de deux menus déroulants : choisissez la **propriété** à gauche, puis **Pick an individual...** à droite.

The screenshot shows the Protege interface for an individual named 'Jane Doe'. The URL is `http://www.w3.org/2002/07/owl#Jane_Doe`. The interface is divided into several sections:

- Annotations:** Shows `rdfs:label` with the value 'Jane Doe' and language 'en (English)'. There is an input field for 'Enter property'.
- Types:** Shows 'Person' selected as the class. There is an input field for 'Enter a class name'.
- Relationships:** Shows 'LOCATED_IN' selected as the property and 'San Francisco' selected as the individual. There is an input field for 'Enter property'.
- Identity:** Shows 'Same as' and 'Different from' options, each with an 'Add an individual...' button.

At the bottom, a summary text reads: 'L'individu Jane Doe : typé Person, avec le fait LOCATED_IN San Francisco'.

Jane Doe + LOCATED_IN + San Francisco crée exactement un triplet :

```
:Jane_Doe :LOCATED_IN :San_Francisco .
```

Le menu déroulant de droite ne liste que les individus qui existent déjà, alors créez l'individu cible avant la relation qui pointe vers lui.

Identité : Same as / Different from

Deux champs en bas :

- **Same as** (`owl:sameAs`) — affirme que cet individu et un autre sont *la même chose du monde réel sous deux noms*. `:NYC owl:sameAs :New_York_City` . Tout ce qui est vrai de l'un est désormais vrai de l'autre. C'est le mécanisme pour réconcilier les doublons que l'extraction a produits depuis des documents différents.
- **Different from** (`owl:differentFrom`) — affirme qu'ils ne sont définitivement *pas* la même chose, même s'ils se ressemblent. Utile pour distinguer deux personnes portant le même nom, et pour empêcher une étape ultérieure de déduplication de les fusionner.

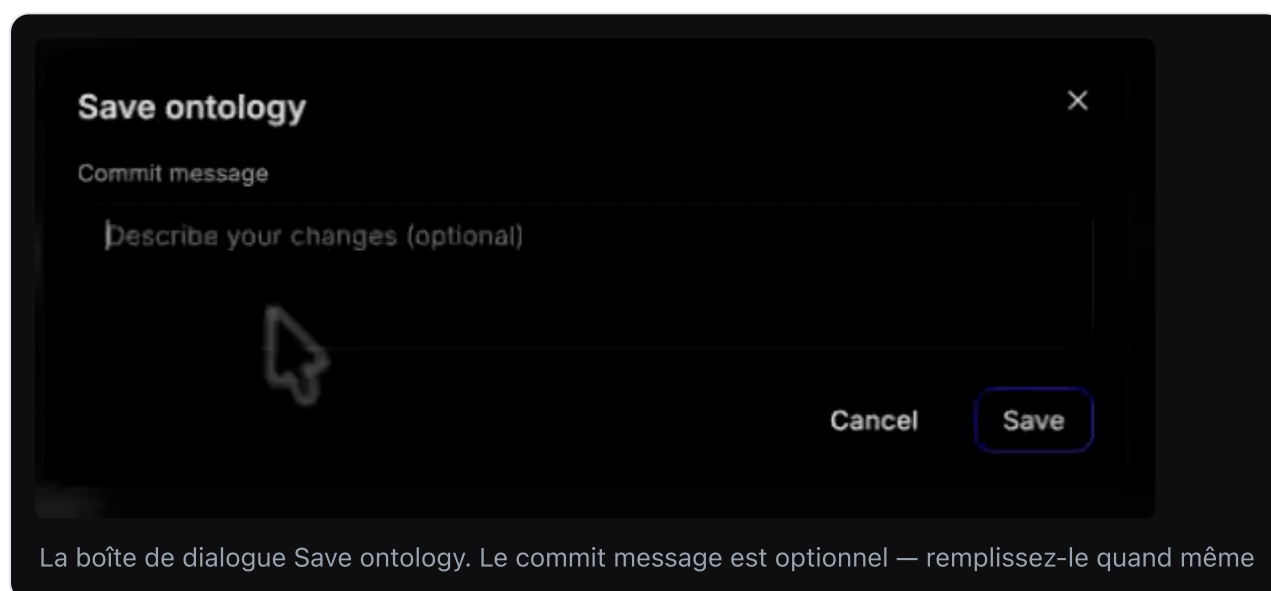
Laissés vides, ni l'un ni l'autre n'est présumé. Deux individus avec des URIs différentes ne sont pas automatiquement considérés comme distincts — c'est l'*hypothèse du monde ouvert* dont RDF hérite, et c'est pour cela que `differentFrom` doit exister.

8. Enregistrement et historique des versions

Perseus versionne les ontologies comme un dépôt git. Chaque enregistrement est un commit avec un hash et un message, et vous pouvez les parcourir en arrière.

Save avec un Commit message

Cliquez sur **Save** (ou ⌘S). Une boîte de dialogue **Save ontology** apparaît avec un champ **Commit message**. Tapez une brève description de ce qui a changé — « *Added Country* », « *People-[LOCATED_IN]→City* » — et cliquez sur **Save**.



La pastille de version dans l'en-tête passe au nouveau hash et conserve le badge **Latest**.

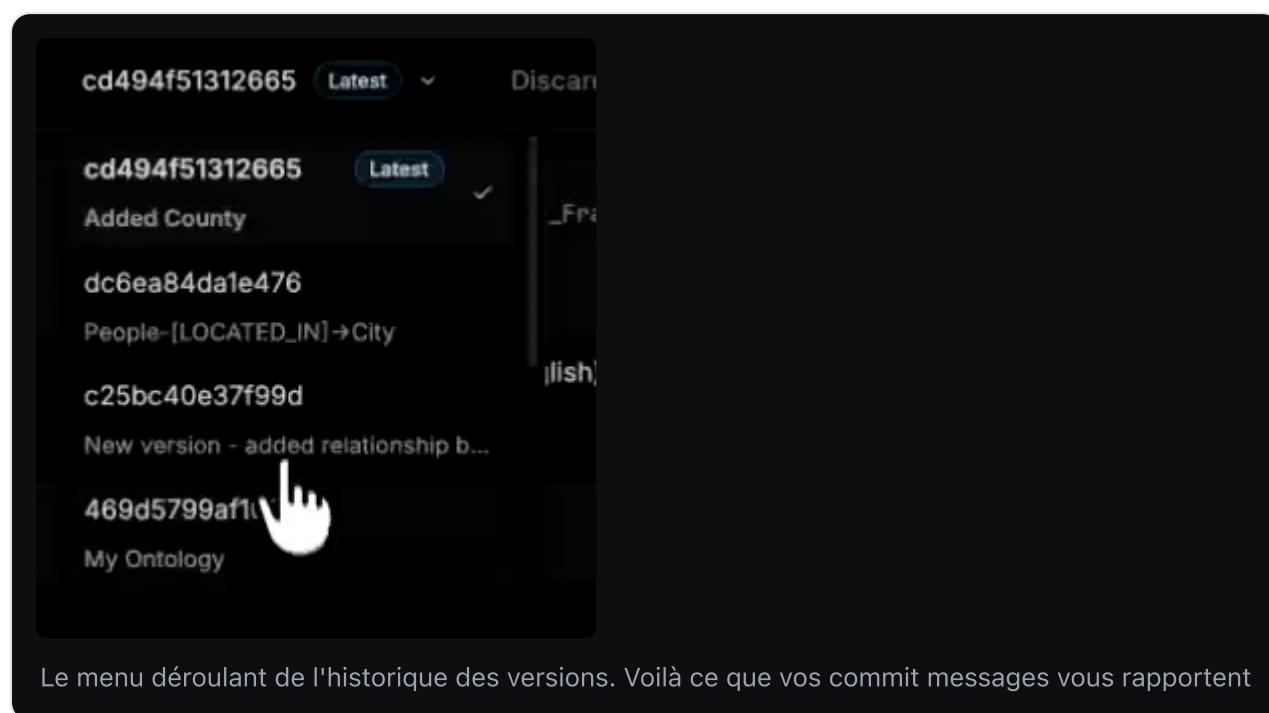
Save est un commit, pas une sauvegarde automatique. Rien n'est persisté tant que vous ne cliquez pas dessus, et quitter la page ou cliquer sur **Discard** fait tout perdre depuis le dernier. Enregistrez tôt, enregistrez souvent — l'historique des versions ne coûte rien, et c'est le seul chemin du retour.

Le champ de message est techniquement **optionnel**, et Perseus committera sans. Remplissez-le quand même. Ces messages *sont* le menu déroulant de l'historique des versions, et une liste de hashes sans étiquette n'est pas un historique navigable.

Parcourir les versions précédentes

Cliquez sur le chevron à côté de la pastille de version. Vous obtenez la liste complète des commits, du plus récent au plus ancien — chaque entrée affichant son hash et son commit message, avec **Latest**

marqué et une coche sur la version que vous consultez actuellement. Sélectionnez n'importe quelle entrée pour charger cette version de l'ontologie.



Le menu déroulant de l'historique des versions. Voilà ce que vos commit messages vous rapportent

Discard

Discard jette toutes les modifications non enregistrées et vous ramène à la dernière version enregistrée. Il ne supprime rien de ce qui a été committé. Il n'y a pas d'annulation pour Discard — mais il y a  pour les erreurs isolées, et c'est en général ce que vous voulez vraiment.

9. Raccourcis clavier

Ouvrez la fenêtre des raccourcis avec l'**icône**  de l'en-tête, ou  .

Shortcuts

Shortcuts details
Open the keyboard shortcuts modal

Undo
Undo the last change

 Z

Redo
Redo the last undone change

  Z

Save
Save the current changes

 S

Focus search
Focus the resource filter input

 G

Create new resource
Create a new resource in the ontology depending on the resource tab.

  N

Edit resource name
Focus the selected resource name input for editing.

F2

Delete resource
Delete the selected resource from ontology.

La fenêtre Shortcuts

Action	Raccourci	Notes
Détail des raccourcis	⌘↑?	Ouvre cette fenêtre.
Annuler	⌘Z	Annule la dernière modification.
Rétablir	⌘↑Z	Rétablit la dernière modification annulée.
Enregistrer	⌘S	Ouvre la boîte de dialogue du commit message.
Aller à la recherche	⌘G	Saute au champ de filtre du panneau de gauche.
Créer une ressource	⌘↑N	Crée une classe, une propriété ou un individu — selon l'onglet où vous êtes.
Renommer la ressource	F2	Place le focus sur le nom de la ressource sélectionnée pour la renommer.
Supprimer la ressource	⌘↑⌫	Supprime de l'ontologie la ressource sélectionnée.

Sur Windows et Linux, remplacez ⌘ par Ctrl .

⌘G → taper → ⌘↑N → taper le nom → F2 est la boucle rapide pour bâtir une hiérarchie sans toucher à la souris.

10. Un exemple complet

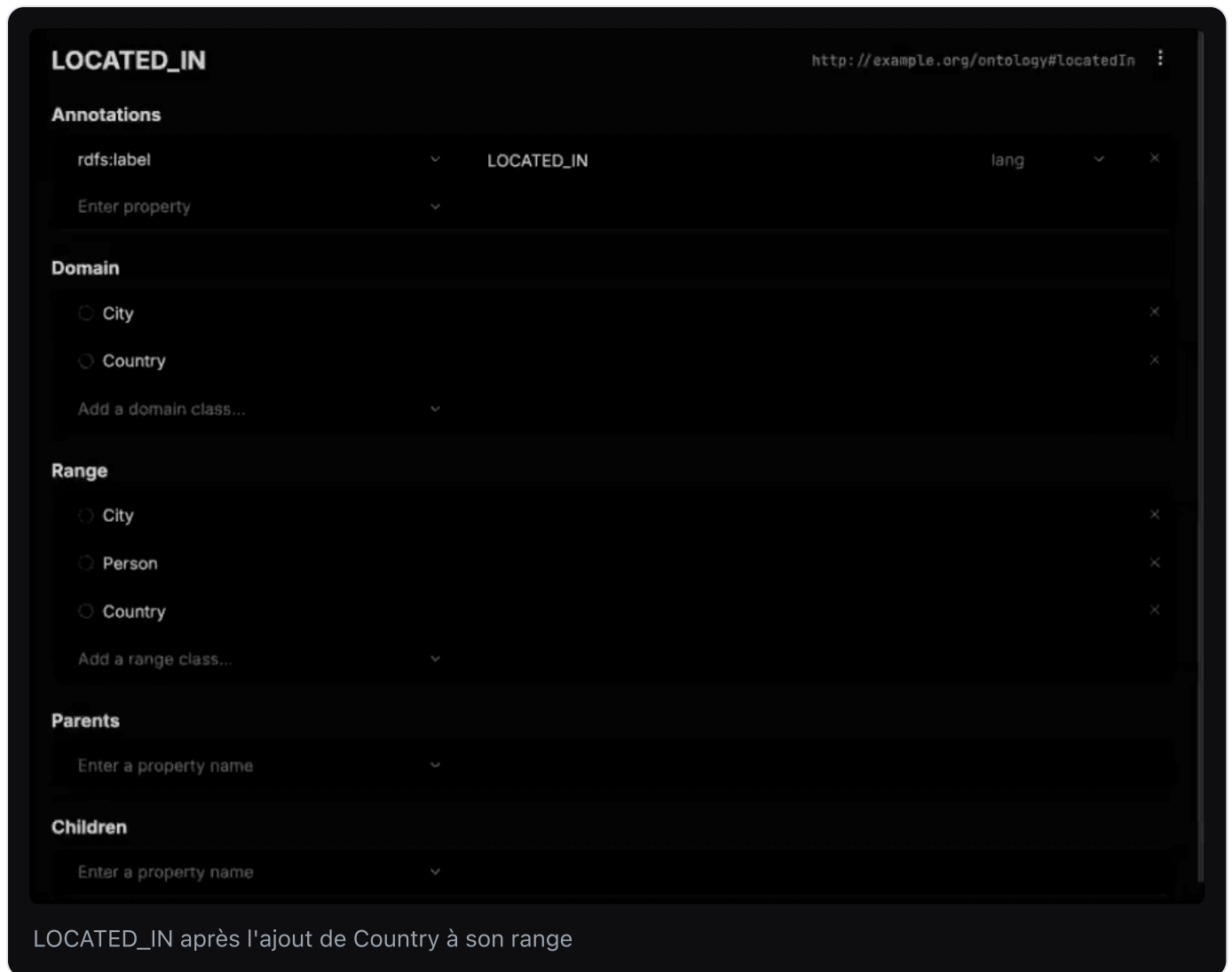
Cet exercice parcourt tout l'éditeur — une classe, une propriété, des individus, des faits, un commit — en six étapes. Ouvrez n'importe quelle ontologie dont vous disposez ; une ontologie générée par **Build Ontology** convient bien, puisqu'elle contient déjà des classes et des propriétés sur lesquelles s'appuyer.

Le parcours ci-dessous s'appuie sur une petite ontologie dont les classes comprennent `City` et `Person` et dont les propriétés d'objet comprennent `LOCATED_IN`. Remplacez-les par vos propres noms au fil de la lecture : la forme de chaque étape est la même quel que soit votre domaine. Le but est d'ajouter une classe manquante, de la brancher sur une relation existante, puis d'énoncer deux faits qui l'utilisent.

1. Ajouter la classe manquante. Onglet **Classes** → `+ Add Class` → la nouvelle classe *Untitled* apparaît. Tapez `Country` par-dessus le titre. Perseus renseigne `rdfs:label = "Country"` avec la langue `en (English)` et fixe l'URI à `owl#Country`.

2. La documenter. Dans **Annotations**, ajoutez une deuxième ligne : `dcterms:description`, avec une phrase expliquant ce qui compte comme un `Country` dans ce domaine.

3. Faire participer la nouvelle classe à une relation. Onglet **Properties** → **Object** → sélectionnez une propriété existante, ici `LOCATED_IN`. Sous **Range**, cliquez sur **Add a range class...** et ajoutez `Country`. Le range est maintenant `{City, Person, Country}` et le domain `{City, Country}` — donc une `City` peut désormais être `LOCATED_IN` un `Country`.



4. Créer deux individus. Onglet **Individuals** → **+ Add Individual** → nommez-le **Jane Doe** → sous **Types**, choisissez **Person**. Recommencez : **+ Add Individual** → **USA** → sous **Types**, choisissez **Country**. Le panneau de gauche montre maintenant chaque individu avec sa classe à côté.

5. Affirmer les faits. En supposant qu'un individu **San Francisco** de classe **City** existe déjà :

- Sélectionnez **Jane Doe** → **Relationships** → propriété **LOCATED_IN**, individu **San Francisco**.
- Sélectionnez **San Francisco** → **Relationships** → propriété **LOCATED_IN**, individu **USA**.

6. Committer. Save → commit message « *Added Country* » → **Save**. La pastille de version passe à un nouveau hash portant le badge **Latest**, et le nouveau commit est en tête du menu déroulant des versions.

En Turtle, ce que vous venez de construire, c'est :

```
:Country      rdf:type      owl:Class ;  
              rdfs:label    "Country"@en .  
  
:LOCATED_IN   rdfs:range    :City, :Person, :Country .  
  
:Jane_Doe     rdf:type      :Person ;  
              :LOCATED_IN   :San_Francisco .  
  
:San_Francisco rdf:type      :City ;  
              :LOCATED_IN   :USA .  
  
:USA          rdf:type      :Country .
```